

1-Number of Zeros in a Given Array

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer:

```
#include<stdio.h>

int main(){

    int n;

    scanf("%d",&n);

    int arr[n];

    for(int i=0;i<n;i++){

        scanf("%d",&arr[i]);

    }

    int low=0;

    int high=n-1;

    int firstZero=n;

    while(low<=high){

        int mid=(low+high)/2;

        if(arr[mid]==0){

            firstZero=mid;

            high=mid-1;

        }

        else{

            low=mid+1;

        }

    }

    int answer=n-firstZero;
```

```

    printf("%d",answer);

    return 0;

}

```

Input	Expected	Got	
5 1 1 1 0 0 0	2	2	
10 1 1 1 1 1 1 1 1 1 1 1 1 1	0	0	
8 0 0 0 0 0 0 0 0 0 0	8	8	
17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	

2-Majority Element

Given an array **nums** of size **n**, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer:

```
#include<stdio.h>

int main(){

    int n;

    scanf("%d",&n);

    int arr[n];

    for(int i=0;i<n;i++){

        scanf("%d",&arr[i]);

    }

    int count=0;

    int candidate=0;

    for(int i=0;i<n;i++){

        if(count==0){
```

```

        candidate=arr[i];
    }
    if(arr[i]==candidate){
        count++;
    }
    else{
        count--;
    }
}

printf("%d",candidate);

return 0;
}

```

Input	Expected	Got	
	3 3 2 3	3	3

3-Finding Floor Value

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array
 Next n lines Contains n numbers – Elements of an array
 Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer:

```
#include<stdio.h>
```

```
int main(){
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```

for(int i=0;i<n;i++){
    scanf("%d",&arr[i]);
}

int x;

scanf("%d",&x);

int low=0;

int high=n-1;

int floor=-1;

while(low<=high){

    int mid=(low+high)/2;

    if(arr[mid]<=x){

        floor=arr[mid];

        low=mid+1;

    }

    else{

        high=mid-1;

    }

}

printf("%d",floor);

return 0;

}

```

Input	Expected	Got	
6 1 2 8 10 12 19 5	2	2	
5 10 22 85 108 129 100	85	85	

Input	Expected	Got	
	7	9	9
	3		
	5		
	7		
	9		
	11		
	13		
	15		
	10		

4-Two Elements sum to x

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer:

```
#include<stdio.h>

int main(){

    int n;

    scanf("%d",&n);

    int arr[n];

    for(int i=0;i<n;i++){

        scanf("%d",&arr[i]);

    }

    int x;

    scanf("%d",&x);

    int found=0;
```

```

for(int i=0;i<n && !found;i++){

    int target=x-arr[i];

    int low=i+1;

    int high=n-1;

    while(low<=high){

        int mid=(low+high)/2;


        if(arr[mid]==target){

            printf("%d\n",arr[i]);

            printf("%d\n",arr[mid]);

            found=1;

            break;

        }

        else if(arr[mid]<target){

            low=mid+1;

        }

        else{

            high=mid-1;

        }

    }

}

if(!found){

    printf("No");

}

return 0;

}

```

Input	Expected	Got	
4	4	4	
2	10	10	
4			
8			

Input	Expected	Got		
	10 14			
	5 2 4 6 8 10 100	No	No	

5-Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```
#include <stdio.h>

int a[100];

int quicksort(int low, int high)
{
    int i, j, pivot, temp;
    if (low >= high)
        return 0;
    i = low;
    j = high;
    pivot = a[low];
    while (i < j)
```

```

{
    while (a[i] <= pivot && i < high)
        i++;
    while (a[j] > pivot)
        j--;
    if (i < j)
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}
a[low] = a[j];
a[j] = pivot;
quicksort(low, j - 1);
quicksort(j + 1, high);
return 0;
}

int main()
{
    int n, i;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    quicksort(0, n - 1);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}

```

Input	Expected	Got	
	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98

Input	Expected	Got	
	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114
	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90